SENC: Handling Self-collision in Neural Cloth Simulation

Zhouyingcheng Liao*, Sinan Wang*, and Taku Komura†

The University of Hong Kong

https://zycliao.github.io/senc

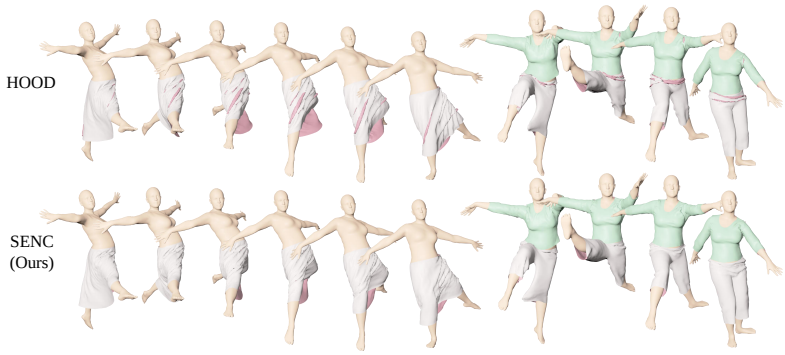

Fig. 1: Our method effectively addresses cloth self-collision, compared to existing state-of-the-art neural cloth simulator [17]. Note the inner side of the cloth is painted pink.

Abstract. We present SENC, a novel self-supervised neural cloth simulator that addresses the challenge of cloth self-collision. This problem has remained unresolved due to the gap in simulation setup between recent collision detection and response approaches and self-supervised neural simulators. The former requires collision-free initial setups, while the latter necessitates random cloth instantiation during training. To tackle this issue, we propose a novel loss based on Global Intersection Analysis (GIA). This loss extracts the volume surrounded by the cloth region that forms the penetration. By constructing an energy based on this volume, our self-supervised neural simulator can effectively address cloth self-collisions. Moreover, we develop a self-collision-aware graph neural network capable of learning to handle self-collisions, even for parts that are topologically distant from one another. Additionally, we introduce an effective external force scheme that enables the simulation to learn the cloth's behavior in response to random external forces. We validate the efficacy of SENC through extensive quantitative and qualitative experiments, demonstrating that it effectively reduces cloth self-collision while maintaining high-quality animation results.

*Equal contribution.
†Corresponding author.

1 Introduction

Self-supervised neural cloth simulation is attractive in the sense that the system can train itself and does not require the user to prepare ground truth garment data, which is usually very expensive to acquire. Among self-supervised techniques, methods that are trained for specific garments [5, 44] or those that can generalize to arbitrary garments exist [17].

Despite the advancements in such self-supervised neural cloth simulation technologies, a critical impediment to the generation of realistic and precise animations remains unaddressed: *cloth self-collision*, which is a very common type of animation artifact that can happen in various conditions, such as the self-penetrations that occur in clothing due to close contact areas, like under the arms when the arms are pressed against the body and overlayed skirts (shown in Figure 1 and Figure 7).

Although a variety of collision detection and response techniques [2, 29, 47, 52] have been proposed for traditional physical-based simulation, they cannot be easily applied to self-supervised neural cloth simulators due to the special treatment needed. For example, the incremental potential contact (IPC) [29], the state-of-the-art collision detection and response technique, uses a barrier method to prevent collisions. It assumes the simulated object starts from a non-colliding state, and the collision energy increases to infinity when the colliding pairs approach each other, which may lead to gradient explosion if used in neural simulation. Such a characteristic makes it unsuitable for neural cloth simulators, which randomly instantiate the state of the garment during the training process. Other techniques that form a penalty function based on edge-edge collisions of two consecutive frames [2, 47] are also not suitable for the same reason, as they also require starting from a collision-free state. Finally, methods based on signed distance functions (SDF) [52] are not suitable for our purposes, as the forces generated by the gradient of the SDF only locally push the particles towards the nearest surface, resulting in local minima. For instance, in cases where one part deeply penetrates another, the middle section may become trapped inside the mesh when the two ends of the penetration volume are pulled in opposite directions.

To overcome these difficulties, we propose SENC, *i.e.*, handling **SE**lf-collision in **N**eural **C**loth simulation, which composes a novel self-collision loss and a self-collision-aware graph neural network. The self-collision loss is based on the volume surrounded by the self-penetrating area of the cloth, which is computed by Global Intersection Analysis (GIA) [1]. The idea of GIA is to analyze the boundaries of self-collisions, i.e., intersection paths, and identify vertices within the intersection paths. The gradient of the volume naturally forms a force that drives the garment vertices in the direction that resolves the collision.

Our method is developed upon the Graph Neural Network [46] (GNN) architecture, which has shown excellent performance in learning cloth dynamics [17, 39, 41], but has failed to consider self-collisions. Previous methods [17, 39, 41] form graphs according to the topological connectivity of the mesh, where the nodes correspond to the vertices of the mesh, and the edges represent their

connections. Although such graphs are effective in predicting the dynamics of clothing based on the local deformation of the fabric caused by the interaction of topologically adjacent areas, these structures cannot be used to prevent cloth self-collision, as many self-collisions happen between areas with a large topological distance. To address this issue, we propose the self-collision-aware GNN, where we construct additional edges based on the spatial distance of vertices, which effectively prevents cloth self-collision. In addition, our model can model variable external forces, allowing the users to provide external forces e.g. based on wind, to increase the dynamic behavior of the garment.

We examine our scheme in various types of garments, including t-shirts, pants, long sleeves, skirts, and dresses. The experiments show that our approach can significantly reduce the amount of self-collisions compared to existing state-of-the-art methods (see Fig. 1).

In summary, the contribution of our paper can be summarized as follows:

- A novel self-collision loss based on GIA that reduces self-collisions on self-supervised neural cloth simulation and
- a self-collision-aware graph neural network that also allows users to apply external forces to the garment.

2 Related Work

In this section, we begin by reviewing cloth simulation, a foundational technique that has laid the groundwork for animating realistic garment deformations. We then explore neural cloth simulation, an innovative approach that has emerged as machine learning techniques have matured. Finally, we delve into collision detection and response techniques, which constitute this research's primary focus.

2.1 Cloth Simulation

Clothes are mostly simulated as a Lagrangian model where mass particles are connected to each other through elastic rods. The large time step is first achieved by using implicit time integration, as proposed in [2], which can reproduce plausible deformation of the clothes with realistic wrinkles and folds. Additionally, various methods based on mass springs [32], finite element models [49], projective dynamics [31], yawn-level models [25, 49], and material point methods [24] have been proposed to realistically simulate the dynamics of the cloth. Despite their precision and the realism they offer, numerical simulation schemes can incur significant computational costs.

2.2 Learning Garment Dynamics

For offline film scenes, expensive numerical simulations are feasible, but there are cases like real-time games that demand faster solutions. Neural cloth simulation meets this need effectively. Some researchers try to learn cloth dynamics from

the pose and/or shape of the 3D human and predict the clothing deformation in the unposed space [20, 33, 38, 43, 60]. To finally deform the garment, these methods rely on linear blend skinning. Thus, they only suit those garments that closely stick to the body. The assumption that the cloth deformation is conditioned on the human pose also makes the result of these papers lack dynamics. DeePSD [6] trains a neural network to predict the cloth skinning weight from the canonical shape. SSCH [45] introduces a diffused human model to project the ground truth cloth data into the canonical space for better learning in the canonical space. These methods enable the deformation of looser garments. However, all these methods require supervised learning with a great amount of ground truth data, and they often fall short of accurately capturing the true physical constraints of garments. In response, self-supervised learning strategies incorporating physical loss functions, such as SNUG [44], ReFU [51], PBNS [4], NCS [5], and GenSim [54] have emerged. These techniques strive to more faithfully mirror the underlying physical principles. Notably, the introduction of graph neural networks by HOOD [17] has demonstrated a robust capability for simulating realistic garment deformations. Methods combining supervised and unsupervised losses like GarSim [53] also appear. Recent diffusion-based methods [59, 61] generate diverse types of garments, but they lack physical plausibility. CaPhy [50] and ULNeF [42] try to resolve collisions between different layers of garments. Yet, a critical issue that persists across all these approaches is their inability to address garment self-collision, a complex challenge that remains unresolved due to various previously outlined factors. ClothCombo [28], a quasistatic multilayer system but not a dynamic animation system, applies a simple repulsion loss to separate close vertices. Such forces cannot resolve self-collisions after it has already occurred. Concurrently, ContourCraft [16] uses the contour length of the self-intersection as the unsupervised loss, which may even increase self-collisions when the true resolving direction is opposite to the direction of reducing the contour length of self-collision. Our method effectively addresses this issue, offering seamless integration into existing neural cloth simulation frameworks, thus advancing the state-of-the-art in realistic neural cloth simulation.

2.3 Collision Detection and Response

Specialized acceleration data structures, such as Bounding Volume Hierarchies (BVH) [13] for spatial division, form the foundation of collision detection techniques. Various algorithms for constructing BVH, including Top-Down [22, 27, 55], Bottom-Up [19, 56], and Incremental [7, 15] construction, together with Linear BVH [26, 37], have been proposed. Additionally, hardware support can be designed to further enhance the speed, as proposed in [10–12]. For more details about BVH and its related work, we refer readers to [35]. For modern cloth simulation methods using implicit time integration, Continuous Collision Detection (CCD) is used. It computes the maximum time step during the line search to prevent collision. Examples include CCD [57] and additive CCD (ACCD) [30].

After the collision has been detected, an appropriate collision response is required to handle and resolve the collision. For those methods using implicit

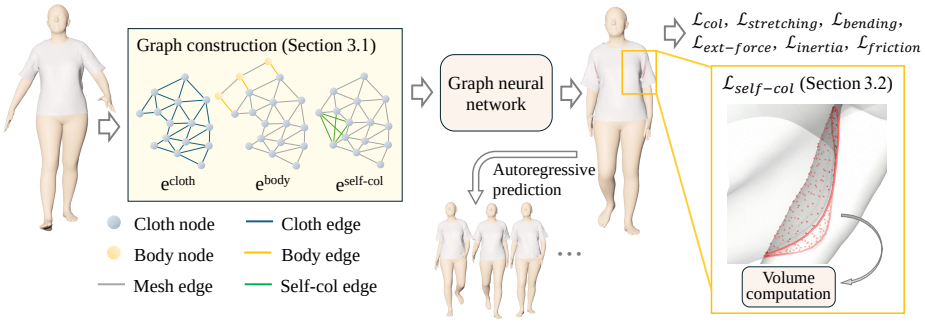

Fig. 2: Method overview.

time integration, the collision can be designed as some penalty energy and integrated into the optimization framework. Its recent representatives are Incremental Potential Contact (IPC) [29] for deformable objects and Codimensional IPC (C-IPC) [30] for cloth-like thin structures. However, as previously mentioned, these approaches primarily aim to prevent collisions before they occur, which does not meet our requirements. Other methods capable of addressing collisions as they occur, such as Untangling Cloth [1], or those involving local adjustments [8,21,40], and with volume-preserving impulses [48], are also not suitable due to their need for additional post-processing, therefore cannot be integrated into the framework of neural cloth simulation.

3 Methodology

We aim to learn a neural model that autoregressively predicts the dynamics of the cloth, which should be physically correct and have minimal collision, including cloth self-collision and cloth-body collision. We address the cloth self-collision issue in neural cloth simulation, by introducing the self-collision-aware network and the self-collision loss. The overview of our method is shown in Fig. 2. In this section, we first describe our self-supervised neural cloth simulator in Section 3.1 and then about the the self-collision loss in Section 3.2, which is the key to handling cloth self-collision in our framework.

3.1 Self-Collision-Aware Neural Cloth Simulator

In this section, we first introduce our problem settings and the models we build upon. Next, we discuss why previous network structures cannot be used to handle the self-collision problem and present the self-collision-aware graph neural network. Finally, we describe our physical energy model and how it is used to supervise the training without ground truth data.

Background Following MeshGraphNets [39] and HOOD [17], our model is a neural cloth simulator that predicts the state of the cloth mesh at time $t+1$

given the current state at t. The cloth and the body are both represented by the mesh $M = \{V, F\}$ with vertices V connected as faces F. A graph neural network, which is agnostic to mesh connectivity, is used to process the cloth and its interaction with the body.

The graph neural network used in MeshGraphNets [39] and HOOD [17] contains two types of features: nodal feature $\mathbf{x}$ and edge feature $\mathbf{e}$. The nodal feature represents the state of vertices. The input nodal features to the network consist of the vertex type (body or cloth), velocity, normal, and physical material properties. The edge feature characterizes the interaction between two vertices. Different types of edges exist, including the cloth-cloth edge $\mathbf{e}^{\text{cloth}}$ and cloth-body edge $\mathbf{e}^{\text{body}}$. The input cloth-cloth edge features consist of the relative position of vertices $\mathbf{v}_i - \mathbf{v}_j$ and the norm $|\mathbf{v}_i - \mathbf{v}_j|$ in both the current and canonical states. The cloth-body edge connects every cloth vertex to the closest body vertex if their distance is below a threshold. Its input features contain the relative position and the distance in the current and the previous frame.

The input nodal and edge features are first embedded into latent vectors, followed by several hierarchical message-passing blocks [17]. In each message-passing, each type of edge features are first independently processed by different networks, and then node features are updated by incorporating its incident edge features:

$$\mathbf{e}'_{ij} \leftarrow f_{\text{edge}}(\mathbf{e}_{ij}, \mathbf{v}_i, \mathbf{v}_j), \qquad \mathbf{x}'_i \leftarrow f_{\text{node}}(\mathbf{x}, \sum_j \mathbf{e}'^{\,\text{body}}_{ij}, \sum_j \mathbf{e}'^{\,\text{cloth}}_{ij}) \tag{1}$$

All networks here are multi-layer perceptrons (MLP). The last layer is an additional MLP that converts latent features into vertex accelerations, from which the vertex positions of the next frame can be obtained using the explicit Euler integration method.

Self-collision-aware Graph Neural Network Although the graph neural network in HOOD [17] is able to produce clothing dynamics with minimal body-cloth collision, it cannot be used to handle cloth self-collision (see Section 4.1). Their networks use cloth mesh connectivity to reconstruct the graph, and the messages are propagated topologically, which is similar to how the local deformation of the cloth propagates across the mesh following its topological connections. However, in many cases, the self-collision occurs between two parts of the cloth that are topologically far (see Figure 5 (c) and (d)).

Based on this observation, we construct additional edges between cloth vertices according to their spatial distances. For each cloth vertex $\mathbf{v}_i$, we search for vertex $\mathbf{v}_j$ so that $\|\mathbf{v}_i - \mathbf{v}_j\| < r$, and construct a self-collision edge $\mathbf{e}^{\text{self-col}}_{ij}$ between them. r is set to 2 cm empirically. Moreover, We exclude all original mesh edges when constructing self-collision edges because if two vertices share a mesh edge, they will not collide with each other. In our ablation study, we found excluding the original mesh edges does not harm the model performance while saving computation.

The self-collision edges are updated similarly to other types of edges. The nodal features update becomes:

$$\mathbf{x}_i' \leftarrow f_{\text{node}}(\mathbf{x}, \sum_j \mathbf{e}_{ij}'^{\text{ body}}, \sum_j \mathbf{e}_{ij}'^{\text{ cloth}}, \sum_j \mathbf{e}_{ij}'^{\text{ self-col}}). \tag{2}$$

Additionally, we model the variable external forces by appending the external force to the input nodal feature. During training in each iteration, we generate a force of random magnitude and direction, add it to the gravity, and set it as the input nodal feature. After training, our model enables the user to provide external forces, such as the wind, for more dynamic behaviors.

Cloth Energy Model The cloth energy model is defined here to reflect the physical properties of the cloth and can be used to supervise the model training without ground-truth data. The self-collision term $\mathcal{L}_{self\text{-}col}$ penalizes the self-penetration volume, therefore preventing self-collision of the garment, as will be explained in Section 3.2. The body-cloth collision term $\mathcal{L}_{col}$ is $\mathbf{max}(\epsilon - \mathbf{SDF}(x), 0)$, which measures the signed distance function for every vertex of the garment with respect to the body mesh, therefore, prevents the collision between the garment and the body, as explained in [45]. The stretching term models the stretching forces with the St.Venant-Kirchhoff material [36]. Similarly, the bending term [18] models the resistance to deformations that attempt to misalign adjacent faces. The external force energy for vertex $\mathbf{v}_i$ is $-\mathbf{q}_i\mathbf{v}_i$, where $\mathbf{q}_i$ is the sum of external forces except contact forces caused by the body. The inertia term [17] represents inertia, which means objects tend to maintain their original velocities. The friction term models the friction forces between the garment and the body, as described in [9,14,17]. The full loss is

$$\begin{aligned}\mathcal{L} = \mathcal{L}_{self\text{-}col}(V^{t+\Delta t}) + \mathcal{L}_{col}(V^t, V^{t+\Delta t}) + \mathcal{L}_{stretching}(V^{t+\Delta t}) + \mathcal{L}_{bending}(V^{t+\Delta t}) \\ + \mathcal{L}_{ext\text{-}force}(V^{t+\Delta t}) + \mathcal{L}_{inertia}(V^{t-\Delta t}, V^t, V^{t+\Delta t}) + \mathcal{L}_{friction}(V^{t+\Delta t}).\end{aligned} \tag{3}$$

3.2 Self-collision Loss

In this section, we explain the process to compute the self-collision loss that is based on the penetration volume that is formed when the garment intersects with itself (see Figure 5). The algorithm is outlined in Alg. 1, where input V and F are the vertices and faces of the garment that we want to compute the self-intersection loss, and $\boldsymbol{\mathcal{B}}$ is the hole boundaries. The process of computing the self-collision loss can be divided into the following steps that we describe next: (1) **garment closure** (steps 2-7), (2) **remeshing** (step 8), (3) **Global Intersection Analysis** (steps 9-12), and (4) **computing the penetration volume** (step 13).

Algorithm 1 Compute the penetration volume

Input: $V, F, \boldsymbol{\mathcal{B}}$ ▷ Vertices and faces of the garment, and the hole boundaries
Output: *volume*

volume ← 0;
$V_{\text{cls}}, F_{\text{cls}} \leftarrow V, F$; ▷ Initialize $V_{\text{cls}}, F_{\text{cls}}$
for each boundary path b in $\boldsymbol{\mathcal{B}}$ **do** ▷ Close the garment
  $v_{\text{central}} \leftarrow$ **MeanBoundary**(b)
  Add v_{central} to V_{cls};
  Connect v_{central} with vertices on b and add faces to F_{cls};
end for
$V_{\text{re}}, F_{\text{re}} \leftarrow$ **Remesh**$(V_{\text{cls}}, F_{\text{cls}})$; ▷ Remesh the garment
$\boldsymbol{\mathcal{P}} \leftarrow$ **FindIntersectionPath**$(V_{\text{re}}, F_{\text{re}})$; ▷ Global Intersection Analysis starts
for each path p in $\boldsymbol{\mathcal{P}}$ **do**
  $\boldsymbol{\mathcal{F}}_{\text{pen}}$ += **ParallelFloodFill**(p, F_{re});
end for
volume ← **ComputeVolume**$(\boldsymbol{\mathcal{F}}_{\text{pen}}, V_{\text{re}})$; ▷ Compute the penetration volume

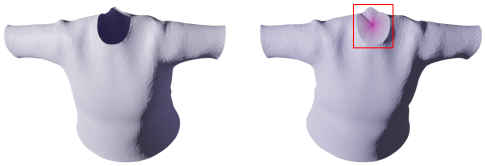

Fig. 3: An example showing how to close the garment.

Garment Closure We first produce a closed surface of the garment by filling in the holes, such as the cuff of the shirt, so that the penetration parts form a closed mesh for computing the volume (see Fig. 3). This is done by simply computing the average position of each hole boundary (represented by v_{central} in Alg. 1) and adding fan triangles that connect the mean point v_{central} and each edge of the hole boundary. This process is conducted for all the hole boundaries (represented by $\boldsymbol{\mathcal{B}}$ in Alg. 1). Regarding garments with more complex geometry, such as shirts with collars, we may choose to leave certain openings unclosed. Despite these remaining openings, our model is capable of learning to eliminate self-intersections from other parts, and still well handles these garments during inference, thanks to the generalization ability of our GNN. Alternatively, a more complex approach is required to define how to close the garment. This method must ensure that the addition of new triangles does not introduce non-existent self-collisions, thereby restricting the garment's movement. Addressing this challenge is designated as part of our future work.

Remeshing Next, given a configuration with self-intersection, we remesh the geometry of the garment so that all intersections lie exactly on the edges of the remeshed garment. The remeshed vertices and faces are represented as V_{re}

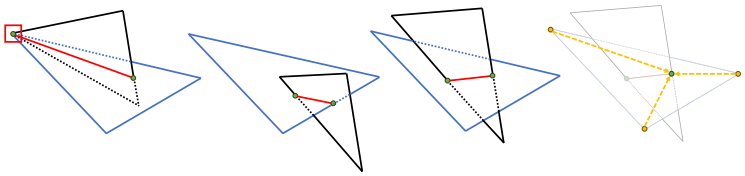

Fig. 4: The leftmost picture shows the case of the loop vertex [1] (enclosed by a red box), where one intersection point is the vertex shared by the two triangles. The middle two pictures show the other cases of two triangles intersecting, generating two intersection points (green circles) respectively. The two green points in these three cases become neighbors in the intersection path (the red line is a segment of the intersection path). These three cases are all the possible cases of two triangles intersecting. The right-most picture shows one intersection point can be represented by the three vertices (yellow circles) of the face using barycentric coordinates.

and F_{re} in Alg. 1. All intersection points are uniquely recorded by $(edge, face)$ pairs, and the exact position of the intersection point can be expressed by the barycentric coordinates of the face. When two triangles intersect, the newly added two intersection points become neighbors in the intersection path, as shown in Figure 4. The detection of triangle-triangle intersection and remeshing process are done using the functions of libiGL library [23].

Global Intersection Analysis (GIA) In this section, we describe how we compute the paths of edges formed by the self-intersection (red lines in Fig. 5, denoted as intersection paths here), and the set of faces surrounded by them. Following [1], we find all intersection paths (represented by $\boldsymbol{\mathcal{P}}$ in Alg. 1) by starting from any $(edge, face)$ intersection pair and tracking its neighbors using the information collected during remeshing. There are two types of self-intersections: those forming only one (Fig. 5(a),(b)) intersection path and those forming two(Fig. 5(c),(d)). The former case happens when a single region has been folded on top of itself while the latter is produced when two distinct regions of the mesh intersect. The key difference between these two cases is the existence of loop vertices. A loop vertex is defined as the vertex shared by two intersecting triangles, as shown in 4. The former can be distinguished from the latter when we find the loop vertices exist in the intersection paths. Loop vertices can be identified during the intersection tests of triangles because we can only find one $(edge, face)$ intersection pair when a loop vertex exists, as shown in the left most case of two triangles intersecting in Figure 4.

Next, the set of faces bound by the intersection path that forms the penetration volume (represented by $\boldsymbol{\mathcal{F}}_{\text{pen}}$ in Alg. 1) are extracted. Since it's topologically ambiguous which set of faces bound by the intersection path is in penetration, e.g., for Figure 5 (b), the one enclosed by the intersection path or the rest of the body. Therefore, a heuristic is formed that the smaller side is the one in penetration. The parallel flood fill algorithm is then used, where we traverse the faces on both sides of the intersection path simultaneously while prohibiting the

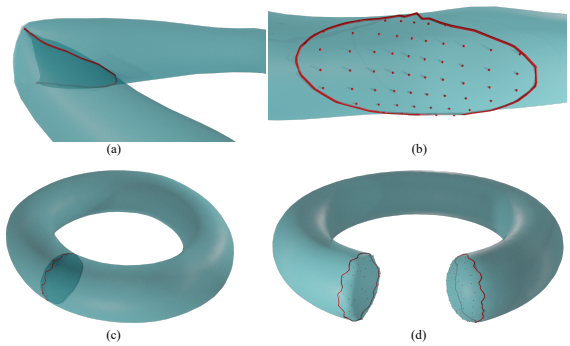

Fig. 5: Here we show two cases of self-collision, where the penetration volume is composed of one (a)(b) and two (c)(d) interaction paths. (a) and (b) show a severely bent elbow with self-collisions happening inside the elbow. Figure (a) shows the intersection path. Figure (b) shows the vertices inside the self-collision and the intersection path after unbending the arm. Similarly, (c) and (d) show a case where a torus intersects with itself, resulting in two intersection paths and two separate penetration surfaces.

traversal through an edge on the intersection path. The side that first finishes the traversal is considered to be in the penetration region. For those intersection paths formed by two distinctive regions (the bottom two in Figure 3), we need to traverse twice to obtain two groups of faces in penetration. For those intersection paths formed by one region (the top two in Figure 3), the penetration faces are enclosed by one intersection path, so one traversal can extract the mesh forming the intersection. We refer readers to [1] for more details of GIA.

Computing the Penetration Volume Using the faces that form the penetration volume extracted by GIA, we compute the penetration volume that is used to compute the self-collision loss in Section 3.1. The volume of the closed mesh can be computed by summing the signed volume, i.e., the scalar triple product of every tetrahedron that is composed of a face in penetration and the origin. Since all the new vertices produced at the remeshing step are represented using barycentric coordinates of the original vertices, the volume loss can then be backpropagated through the neural network.

4 Experiments

Implementation Details Following [17, 44], we use 52 human motion capture sequences from the AMASS [34] dataset for training. Our training consists of two phases: pre-training without self-collision loss and full training with all losses. At the beginning of training, the network output is very noisy, and conducting GIA for such data is very time-consuming. Numerical errors could also happen during the process of remeshing and GIA, especially when numerous triangles

	t-shirt			skirt		
	$\mathcal{L}_{self\text{-}col}$ ($\times 10^{-3}$) $\downarrow$	% (0.1) $\downarrow$	% (0.01) $\downarrow$	$\mathcal{L}_{self\text{-}col}$ ($\times 10^{-3}$) $\downarrow$	% (0.1) $\downarrow$	% (0.01) $\downarrow$
SNUG	52.00	24.78	38.76	N/A	N/A	N/A
NCS	75.05	33.24	70.58	476.03	99.91	100
HOOD	26.10	8.92	22.62	7.33	1.52	9.84
w. area loss	28.81	8.18	39.08	8.80	1.93	13.61
w/o self-col edge	**1.14**	**0**	**1.70**	7.92	1.61	12.69
w. mesh edge	2.37	**0**	5.47	2.02	**0.14**	3.95
SENC	1.80	**0**	3.724	**1.59**	0.28	**2.71**

Table 1: Quantitative comparison with SOTA methods (upper part) and ablation study (lower part).

gather together. Thus, we pre-train the network without the self-collision loss for 120,000 iterations and then train it for 70,000 iterations with all losses. The whole training process takes around 48 hours on a single Nvidia RTX 4090. We borrow several training techniques from HOOD [17], including initializing garment and autoregressive training.

Competing Methods We compare SENC with several recent representative neural cloth simulation methods. **SNUG** [44] is one of the pioneering works that learn cloth dynamics in a self-supervised manner. It prevents body-cloth collision by introducing a collision loss based on the human body SDF. However, since the authors do not release the training code, we can only use their pre-trained models for certain types of garments. Similarly, **NCS** [5] is trained in a self-supervised manner, while they claim to have better dynamics than previous works. **HOOD** [17] is the first self-supervised neural cloth simulator to model clothing dynamics for arbitrary mesh topology and connectivity. However, none of them handles cloth self-collision, which leads to a great number of artifacts.

4.1 Quantitative Evaluation

As in [44], We evaluate our model on 4 test sequences from AMASS [34] containing 2175 frames, which are unseen during training. Since SNUG and NCS do not support physical material control, we set the same set of fabric materials for all experiments. We analyze the cloth self-collision using the average self-collision loss $\mathcal{L}_{self\text{-}col}$ computed over all test frames. In addition, we calculate the percentage of frames whose $\mathcal{L}_{self\text{-}col}$ is above a certain threshold. We use %(0.1) and %(0.01) to denote the percentages with threshold of 0.1 and 0.01 respectively.

In Table 1, we first compare our method with SNUG, NCS, and HOOD. It can be seen clearly that our method outperforms all other methods by a great margin. As the authors of SNUG did not release the training code, and they do not have the checkpoint for the skirt, we skip its evaluation on the skirt.

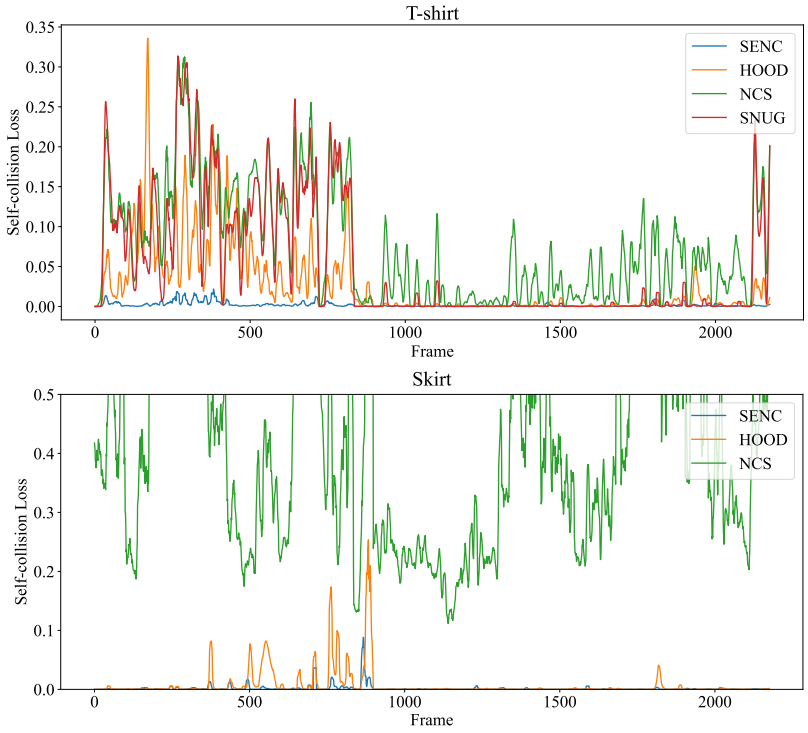

Fig. 6: Quantitative comparisons showing the self-collision loss of different methods on the test sequences. Our method (SENC) shows a significantly lower loss.

Figure 6 visualizes the self-collision losses on the test sequences. This demonstrates that our method successfully addresses the issue of cloth self-collision, in contrast to other approaches that frequently result in such problems.

Ablation Study We further conduct an ablation study to validate the effectiveness of our designs. **w. area loss** denotes that we compute the area of the intersection part (summing all the areas of the faces inside the penetration) instead of the volume. **w/o self-col edge** is the model without constructing the self-collision edges in Equation 2. **w. mesh edge** means we keep all edges, including mesh edges when constructing the self-collision edges. In contrast, our final model excludes original mesh edges when constructing the self-collision edges, because if two vertices are neighbors (connected by a mesh edge), they will not collide with each other.

From Table 1, we can see that if we use the area (**w. area loss**) instead of the volume, the model can hardly learn to handle self-collision. This could be because the gradient to reduce the area of the penetration surface does not effectively direct the vertices to resolve the penetration. **w/o self-col edge** well addresses the self-collision for the t-shirt. However, it fails to handle the skirt.

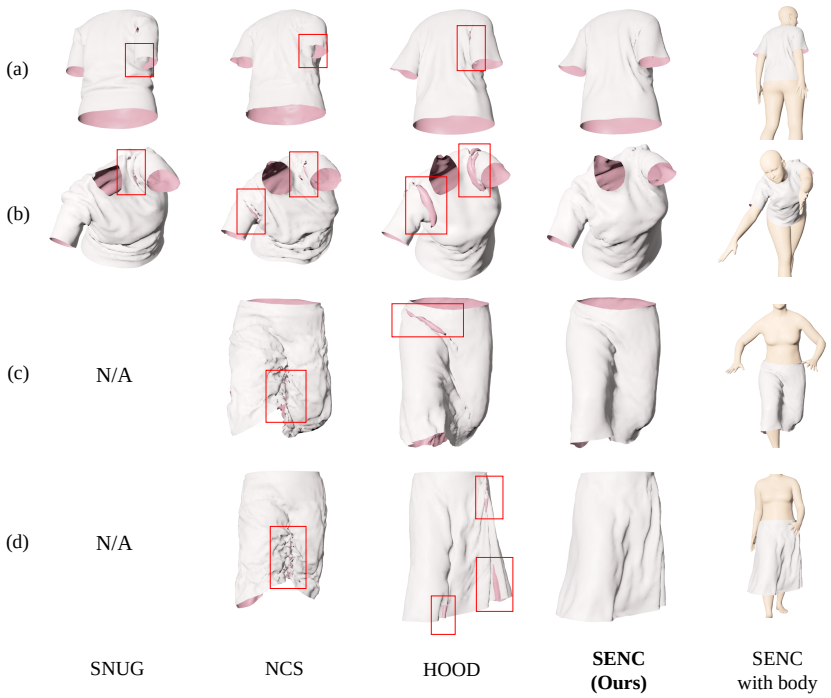

Fig. 7: Comparisons with existing methods. Other methods exhibit clear cloth self-collision, while our method addresses it well.

Compared to the t-shirt, the collision part of the skirt has larger topological distances in the mesh graph. For example, the front and rear hem, which are far topologically from the skirt, could easily collide. With the help of self-collision edges, such self-collision can be well addressed, leading to a lower self-collision loss of our final method. **w. mesh edge** has a slightly worse performance than our final method, showing that including the original mesh edges in self-collision edges does not boost the performance but only introduces extra computation.

4.2 Qualitative Evaluation

We visualize the results of our method and other competing methods in Figure 1 and Figure 7. To enhance the clarity of our visual results, we render the outer side of the garments in white and the inner side in pink. In Figure 7 (a), SNUG and NCS both have a severe self-collision, with a notable portion of the t-shirt sleeve intruding into the torso region of the garment. In (b), all competing methods have a clear self-collision around the shoulder. HOOD has a much larger penetration volume than SNUG and NCS as it is a GNN-based model and has better dynamics. Our method, despite also being GNN-based, avoids the penetration successfully. In (c) and (d), NCS produces a great number of

Fig. 8: Our method enables variable external forces on the cloth. Here the dress is depicted responding to wind from various directions, with the leftmost one not affected by external forces except gravity.

artifacts, as it is a skinning-based pose-dependent method. Similarly, HOOD is still not immune to noticeable self-collision problems.

In Figure 8, we demonstrate that our model enables the user to apply a variable external force on the cloth. The cloth deforms naturally since we jointly optimize the energy by the external force and other energies.

5 Conclusion

We propose a self-supervised scheme for Neural Cloth Simulation, which solves the persisting problem in the literature: self-collision. Based on GIA proposed in [1], we have developed a new self-collision loss, which can be easily integrated into any existing neural cloth simulation framework, without sacrificing the high quality of garment dynamic shown in the state-of-the-art [17].

Limitations and Future Work While our method is able to address self-collisions effectively, it still has some limitations. Firstly, computing the penetration volume is time-consuming, due to bottlenecks caused by remeshing and GIA. Better algorithms for accelerating these processes are a possible future research direction. Secondly, we are only able to handle meshes that can be easily closed. For garments with complex topology, it may be difficult to define how to close the garment. The dilemma is described as follows. Take the long skirt as an example: closing the garment can sometimes inhibit valid deformation. If the bottom hole is closed, then our model will prevent the bottom part of the skirt from moving upwards, as it can result in producing penetration volume between the virtually added triangles and the original skirt mesh. Conversely, leaving it open means the model cannot learn from the cases when self-collisions happen between the front and back hems, as these penetrations will not be closed without sealing the bottom hole. A penetration loss that can roughly compute the bounded volume without closing the boundary is desired: we can possibly apply winding numbers [3] or electronic flux [58] for this purpose. Moreover, our method can generalize to other objects such as multi-layer cloth and 3D deformable characters: extending our method to such topics would be promising.

Acknowledgement

This work is partly supported by the Innovation and Technology Commission of the HKSAR Government under the ITSP-Platform grant (Ref: ITS/335/23FP) and the InnoHK initiative (TransGP project), The research work was in part conducted in the JC STEM Lab of Robotics for Soft Materials funded by The Hong Kong Jockey Club Charities Trust.

SENC: Handling Self-collision in Neural Cloth Simulation –Appendix–

A Repulsive Loss

	t-shirt			skirt		
	$\mathcal{L}_{self\text{-}col}$ $(\times 10^{-3})\downarrow$	% (0.1) $\downarrow$	% (0.01) $\downarrow$	$\mathcal{L}_{self\text{-}col}$ $(\times 10^{-3})\downarrow$	% (0.1) $\downarrow$	% (0.01) $\downarrow$
repulsive loss	15.16	3.31	29.93	22.38	2.30	6.21
SENC	1.80	0	3.724	1.59	0.28	2.71

Table 2: Comparison with the repulsive loss.

We further explore a possible solution, the repulsive loss [28], to handle cloth self-collision. It tries to separate non-adjacent cloth vertices when they are close, and can be formulated as:

$$\mathcal{L}_{repulsive} = \sum_{i}^{N} \sum_{j \in \mathcal{A}_i} -\log(\mathbf{v}_i - \mathbf{v}_j)^2, \tag{4}$$

where $\mathcal{A}_i = \{j \in V \mid (\mathbf{v}_i, \mathbf{v}_j) \notin E \text{ and } d(\mathbf{v}_i, \mathbf{v}_j) < \text{threshold}\}$, and $d()$ is the distance function. We set threshold = 5cm.

Evaluated on the same sequences in Section 5 of our paper, the results in the table above additionally shows the comparisons between ours and the repulsive loss applied on HOOD [17]'s model. For each type of the garment (t-shirt or skirt), the left most column shows directly the averaged self-collision during the evaluation; the middle column shows the proportions of the frames whose self-collisions are higher than 0.1; Similarly, this threshold is set to 0.01 in the right column. It can be seen that ours outperforms the repulsive loss by a large margin.

One of the main reasons for its poor performance is that the repulsive loss simply prevents all vertices pairs from getting too close. Consequently, if self-collisions already exist, then it will also prevent penetrations from being resolved because it does not allow vertices in penetrations to approach the penetration surfaces from where they originally penetrated in.

B More Quantitative Results

In Table 3, we additionally show results on six more unseen garments from the test set of HOOD, where we present the self-collision loss of HOOD and our model on the test sequences. Our model resolves self-collision significantly for all types of garments, further verifying the generalization ability and efficacy of our method.

	pants shorter	short sleeve	tshirt dynamic	novel tank	hooded dress	tight dress
HOOD	6.08	60.66	164.23	6.73	33.37	35.86
SENC	**0.25**	**2.90**	**6.58**	**0.13**	**2.63**	**1.56**

Table 3: Comparison of $\mathcal{L}_{\text{col}}$ on more unseen garments

	t-shirt	skirt	dress	average
HOOD	23.127	20.012	15.638	19.090
w. mesh edge	20.041	16.986	13.078	16.196
SENC	20.179	17.942	13.732	16.843

Table 4: Runtime speed (unit: frame per second).

C Runtime Speed

We measure the runtime speed of our method and compare it with HOOD [17] and **w. mesh edge**, an ablation setting where the mesh edge is not excluded when constructing self-collision edges (More details in the main paper). The speed of our method is slightly slower than HOOD due to the construction of self-collision edges. However, our self-collision is significantly reduced compared to HOOD. Compared to **w. mesh edge**, our final method has a faster speed and better self-collision prevention (See Table 1 of the main paper), which validates the effectiveness of our design.

References

1. Baraff, D., Witkin, A., Kass, M.: Untangling cloth. ACM Transactions on Graphics (TOG) **22**(3), 862–870 (2003) 2, 5, 9, 10, 14
2. Baraff, D., Witkin, A.P.: Large steps in cloth simulation. In: Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1998, Orlando, FL, USA, July 19-24, 1998. pp. 43–54. ACM (1998). https://doi.org/10.1145/280814.280821 2, 3
3. Barill, G., Dickson, N., Schmidt, R., Levin, D.I., Jacobson, A.: Fast winding numbers for soups and clouds. ACM Transactions on Graphics (2018) 14
4. Bertiche, H., Madadi, M., Escalera, S.: Pbns: physically based neural simulator for unsupervised garment pose space deformation. arXiv preprint arXiv:2012.11310 (2020) 4
5. Bertiche, H., Madadi, M., Escalera, S.: Neural cloth simulation. ACM Transactions on Graphics (TOG) **41**(6), 1–14 (2022) 2, 4, 11
6. Bertiche, H., Madadi, M., Tylson, E., Escalera, S.: Deepsd: Automatic deep skinning and pose space deformation for 3d garment animation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 5471–5480 (2021) 4
7. Bittner, J., Hapala, M., Havran, V.: Incremental bvh construction for ray tracing. Computers & Graphics **47**, 135–144 (2015) 4
8. Bridson, R., Fedkiw, R., Anderson, J.: Robust treatment of collisions, contact and friction for cloth animation. In: Proceedings of the 29th annual conference on Computer graphics and interactive techniques. pp. 594–603 (2002) 5
9. Brown, G.E., Overby, M., Forootaninia, Z., Narain, R.: Accurate dissipative forces in optimization integrators. ACM Transactions on Graphics (TOG) **37**(6), 1–14 (2018) 7
10. Doyle, M.J., Fowler, C., Manzke, M.: Hardware accelerated construction of sah-based bounding volume hierarchies for interactive ray tracing. In: Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games. pp. 209–209 (2012) 4
11. Doyle, M.J., Fowler, C., Manzke, M.: A hardware unit for fast sah-optimised bvh construction. ACM Transactions on Graphics (TOG) **32**(4), 1–10 (2013) 4
12. Doyle, M.J., Tuohy, C., Manzke, M.: Evaluation of a bvh construction accelerator architecture for high-quality visualization. IEEE Transactions on Multi-Scale Computing Systems **4**(1), 83–94 (2017) 4
13. Ericson, C.: Real-time collision detection. Crc Press (2004) 4
14. Geilinger, M., Hahn, D., Zehnder, J., Bächer, M., Thomaszewski, B., Coros, S.: Add: Analytically differentiable dynamics for multi-body systems with frictional contact. ACM Transactions on Graphics (TOG) **39**(6), 1–15 (2020) 7
15. Goldsmith, J., Salmon, J.: Automatic creation of object hierarchies for ray tracing. IEEE Computer Graphics and Applications **7**(5), 14–20 (1987) 4
16. Grigorev, A., Becherini, G., Black, M.J., Hilliges, O., Thomaszewski, B.: Contourcraft: Learning to resolve intersections in neural multi-garment simulations. arXiv preprint arXiv:2405.09522 (2024) 4
17. Grigorev, A., Black, M.J., Hilliges, O.: Hood: Hierarchical graphs for generalized modelling of clothing dynamics. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 16965–16974 (2023) 1, 2, 4, 5, 6, 7, 10, 11, 14, 16, 17

18. Grinspun, E., Hirani, A.N., Desbrun, M., Schröder, P.: Discrete shells. In: Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation. pp. 62–67. Citeseer (2003) 7
19. Gu, Y., He, Y., Fatahalian, K., Blelloch, G.: Efficient bvh construction via approximate agglomerative clustering. In: Proceedings of the 5th High-Performance Graphics Conference. pp. 81–88 (2013) 4
20. Guan, P., Reiss, L., Hirshberg, D.A., Weiss, A., Black, M.J.: Drape: Dressing any person. ACM Transactions on Graphics (ToG) **31**(4), 1–10 (2012) 4
21. Harmon, D., Vouga, E., Tamstorf, R., Grinspun, E.: Robust treatment of simultaneous collisions. In: ACM SIGGRAPH 2008 papers, pp. 1–4 (2008) 5
22. Hendrich, J., Meister, D., Bittner, J.: Parallel bvh construction using progressive hierarchical refinement. In: Computer Graphics Forum. vol. 36, pp. 487–494. Wiley Online Library (2017) 4
23. Jacobson, A., Panozzo, D., et al.: libigl: A simple C++ geometry processing library (2018), https://libigl.github.io/ 9
24. Jiang, C., Gast, T., Teran, J.: Anisotropic elastoplasticity for cloth, knit and hair frictional contact. ACM Transactions on Graphics (TOG) **36**(4), 1–14 (2017) 3
25. Kaldor, J.M., James, D.L., Marschner, S.: Simulating knitted cloth at the yarn level. In: ACM SIGGRAPH 2008 papers, pp. 1–9 (2008) 3
26. Karras, T.: Maximizing parallelism in the construction of bvhs, octrees, and k-d trees. In: Proceedings of the Fourth ACM SIGGRAPH/Eurographics conference on High-Performance Graphics. pp. 33–37 (2012) 4
27. Lauterbach, C., Garland, M., Sengupta, S., Luebke, D., Manocha, D.: Fast bvh construction on gpus. In: Computer Graphics Forum. vol. 28, pp. 375–384. Wiley Online Library (2009) 4
28. Lee, D., Kang, H., Lee, I.K.: Clothcombo: Modeling inter-cloth interaction for draping multi-layered clothes. ACM Transactions on Graphics (TOG) **42**(6), 1–13 (2023) 4, 16
29. Li, M., Ferguson, Z., Schneider, T., Langlois, T.R., Zorin, D., Panozzo, D., Jiang, C., Kaufman, D.M.: Incremental potential contact: intersection-and inversion-free, large-deformation dynamics. ACM Trans. Graph. **39**(4), 49 (2020) 2, 5
30. Li, M., Kaufman, D.M., Jiang, C.: Codimensional incremental potential contact. arXiv preprint arXiv:2012.04457 (2020) 4, 5
31. Li, Y., Du, T., Wu, K., Xu, J., Matusik, W.: Diffcloth: Differentiable cloth simulation with dry frictional contact. ACM Trans. Graph. **42**(1) (oct 2022). https://doi.org/10.1145/3527660 3
32. Liu, T., Bargteil, A.W., O'Brien, J.F., Kavan, L.: Fast simulation of mass-spring systems. ACM Transactions on Graphics **32**(6), 209:1–7 (Nov 2013), http://cg.cis.upenn.edu/publications/Liu-FMS, proceedings of ACM SIGGRAPH Asia 2013, Hong Kong 3
33. Ma, Q., Yang, J., Ranjan, A., Pujades, S., Pons-Moll, G., Tang, S., Black, M.J.: Learning to dress 3d people in generative clothing. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 6469–6478 (2020) 4
34. Mahmood, N., Ghorbani, N., Troje, N.F., Pons-Moll, G., Black, M.J.: Amass: Archive of motion capture as surface shapes. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 5442–5451 (2019) 10, 11
35. Meister, D., Ogaki, S., Benthin, C., Doyle, M.J., Guthe, M., Bittner, J.: A survey on bounding volume hierarchies for ray tracing. In: Computer Graphics Forum. vol. 40, pp. 683–712. Wiley Online Library (2021) 4

36. Montes, J., Thomaszewski, B., Mudur, S., Popa, T.: Computational design of skintight clothing. ACM Transactions on Graphics (TOG) **39**(4), 105–1 (2020) 7
37. Pantaleoni, J., Luebke, D.: Hlbvh: Hierarchical lbvh construction for real-time ray tracing of dynamic geometry. In: Proceedings of the Conference on High Performance Graphics. pp. 87–95 (2010) 4
38. Patel, C., Liao, Z., Pons-Moll, G.: Tailornet: Predicting clothing in 3d as a function of human pose, shape and garment style. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 7365–7375 (2020) 4
39. Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., Battaglia, P.: Learning mesh-based simulation with graph networks. In: International Conference on Learning Representations (2020) 2, 5, 6
40. Provot, X.: Collision and self-collision handling in cloth model dedicated to design garments. In: Computer Animation and Simulation'97: Proceedings of the Eurographics Workshop in Budapest, Hungary, September 2–3, 1997. pp. 177–189. Springer (1997) 5
41. Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J., Battaglia, P.: Learning to simulate complex physics with graph networks. In: International conference on machine learning. pp. 8459–8468. PMLR (2020) 2
42. Santesteban, I., Otaduy, M., Thuerey, N., Casas, D.: Ulnef: Untangled layered neural fields for mix-and-match virtual try-on. Advances in Neural Information Processing Systems **35**, 12110–12125 (2022) 4
43. Santesteban, I., Otaduy, M.A., Casas, D.: Learning-based animation of clothing for virtual try-on. In: Computer Graphics Forum. vol. 38, pp. 355–366. Wiley Online Library (2019) 4
44. Santesteban, I., Otaduy, M.A., Casas, D.: Snug: Self-supervised neural dynamic garments. ieee. In: CVF Conference on Computer Vision and Pattern Recognition (CVPR). vol. 2 (2022) 2, 4, 10, 11
45. Santesteban, I., Thuerey, N., Otaduy, M.A., Casas, D.: Self-supervised collision handling via generative 3d garment models for virtual try-on. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 11763–11773 (2021) 4, 7
46. Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G.: The graph neural network model. IEEE transactions on neural networks **20**(1), 61–80 (2008) 2
47. Shi, A., Kim, T.: A unified analysis of penalty-based collision energies. Proceedings of the ACM on Computer Graphics and Interactive Techniques **6**(3), 1–19 (2023) 2
48. Sifakis, E., Marino, S., Teran, J.: Globally coupled collision handling using volume preserving impulses. In: Symposium on Computer Animation. pp. 147–153 (2008) 5
49. Sperl, G., Narain, R., Wojtan, C.: Homogenized yarn-level cloth. ACM Transactions on Graphics (TOG) **39**(4) (2020) 3
50. Su, Z., Hu, L., Lin, S., Zhang, H., Zhang, S., Thies, J., Liu, Y.: Caphy: Capturing physical properties for animatable human avatars. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 14150–14160 (2023) 4
51. Tan, Q., Zhou, Y., Wang, T., Ceylan, D., Sun, X., Manocha, D.: A repulsive force unit for garment collision handling in neural networks. In: European Conference on Computer Vision. pp. 451–467. Springer (2022) 4

52. Teran, J., Sifakis, E., Irving, G., Fedkiw, R.: Robust quasistatic finite elements and flesh simulation. In: Proceedings of the 2005 ACM SIGGRAPH/Eurographics symposium on Computer animation. pp. 181–190 (2005) 2
53. Tiwari, L., Bhowmick, B.: Garsim: Particle based neural garment simulator. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. pp. 4472–4481 (2023) 4
54. Tiwari, L., Bhowmick, B., Sinha, S.: Gensim: Unsupervised generic garment simulator. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4169–4178 (2023) 4
55. Wald, I.: On fast construction of sah-based bounding volume hierarchies. In: 2007 IEEE Symposium on Interactive Ray Tracing. pp. 33–40. IEEE (2007) 4
56. Walter, B., Bala, K., Kulkarni, M., Pingali, K.: Fast agglomerative clustering for rendering. In: 2008 IEEE Symposium on Interactive Ray Tracing. pp. 81–86. IEEE (2008) 4
57. Wang, B., Ferguson, Z., Schneider, T., Jiang, X., Attene, M., Panozzo, D.: A large-scale benchmark and an inclusion-based algorithm for continuous collision detection. ACM Transactions on Graphics (TOG) **40**(5), 1–16 (2021) 4
58. Wang, H., Sidorov, K.A., Sandilands, P., Komura, T.: Harmonic parameterization by electrostatics. ACM Transactions on Graphics (TOG) **32**(5), 1–12 (2013) 14
59. Wang, J., Liu, Y., Dou, Z., Yu, Z., Liang, Y., Li, X., Wang, W., Xie, R., Song, L.: Disentangled clothed avatar generation from text descriptions. In: ECCV (2024) 4
60. Wang, T.Y., Shao, T., Fu, K., Mitra, N.J.: Learning an intrinsic garment space for interactive authoring of garment animation. ACM Transactions on Graphics (TOG) **38**(6), 1–12 (2019) 4
61. Yu, Z., Dou, Z., Long, X., Lin, C., Li, Z., Liu, Y., Müller, N., Komura, T., Habermann, M., Theobalt, C., et al.: Surf-d: High-quality surface generation for arbitrary topologies using diffusion models. In: ECCV (2024) 4